

```

rockstar@supernatural:~/get_started$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help json' for definitive documentation on these fields
and exactly what they do.

Use 'npm install --save' afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (get_started)
version: (1.0.0)
description:
entry point: (index.js)
git repository:
keywords:
author:
license: (ISC)
about to write to /home/rockstar/get_started/package.json:

{
  "name": "get_started",
  "scripts": {
    "flow": "flow test $? -eq 0 || $? -eq 2"
  },
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {},
  "devDependencies": {
    "flow-bin": "^0.8.0"
  },
  "license": "ISC",
  "description": ""
}

Is this ok? (yes)

```

Figure 1: Initialising npm

```

$ mkdir get_started
$ cd get_started
$ npm init
$ npm install --save --save-dev flow-bin
$ npm install --save-dev flow-bin

```

Figure 2: Setting up a sample project

operation, the arguments, expressions and variables must be given a data type. Hence, this helps to eliminate certain classes of errors before the program runs.

Flow is an open source static type checker. It helps to catch common bugs in JavaScript before they run. These bugs include:

1. Dereferencing a null pointer
2. Silent type conversions
3. The dreaded 'undefined is not a function' error message

When Flow is initiated, it performs an initial analysis of all the files present in the code base. After the analysis, it stores the result in a persistent server. When the user saves a file, Flow rechecks all the changes in the background. Flow comes with a lot of new features, some of which are listed here.

Speed: The programmer does not have to wait for Flow to check the code because both the initial analysis and recheck are heavily optimised for performance.

Safety: Flow is designed to find errors. It uses control flow analysis to deeply understand your code to find errors that other type systems can't.

Idiomatic: Flow is exclusively for Java programmers. So, coding with Flow would be like coding using the common idioms in the language.

Gradual: Flow follows the opt-in style of checking, which means that you can gradually convert your existing code base written in JavaScript to Flow on a per file basis. This can be done by adding a `/* @flow */` comment to the top of your source file.

Figure 1 demonstrates how to set up a sample project with npm, which is a package manager for JavaScript. To

```

// @flow
var str = 'test run';
console.log(str);

```

Figure 3: Creating `index.js`

```

$> npm run-script flow
$
> test $? flow /get_started
> flow
No errors!

```

Figure 4: Compiling `index.js` with Flow

```

// @flow
var str: number = 'test run';
console.log(str);

```

Figure 5: Modified `index.js`

```

$> npm run-script flow
> test $? flow /get_started
> flow > /dev/null

index.js:3
  3: let str: number = 'test run';
    ~~~~~ string. This type is incompatible with
  3: let str: number = 'test run';
    ~~~~~ number

Found 1 error

```

Figure 6: Testing with Flow

initialise a project with npm, we can run `npm init` inside our sample project directory. It creates a `package.json` file with the options that you have selected.

Another way of doing this is by creating a sample project directory `get_started` and adding properties like the name and scripts to the `package.json` file. Then add Flow to the project (Figure 2).

Figure 3 shows how to create a file `index.js`.

`// @flow` at the beginning of the file tells Flow to check this file. If this line is neglected, then Flow will not type check this file.

To run Flow

`No errors` indicates that the file `index.js` has been type checked and run error-free. To study the functioning of this static type checker, let's make an error, type check it and see if Flow detects it or not (see Figure 4).

So, now the `index.js` looks like what's shown in Figure 5.

Note the type change we have made here. Now, on compilation, the result looks like what can be seen in Figure 6.

We see that Flow detects the type error: `string. This type is incompatible with number`.

By: H. Chaithanya Krishnan

The author is an open source enthusiast. She is an active contributor to the world of open source. She can be reached at chaitanya7991@gmail.com